

Relatório de Utilização de IA

Resumo dos usos específicos

Uso	Parte do projeto	Resultado gerado	Validação feita
1	Razor Pages	Tela de clientes com consulta, cadastro, atualização e popup de confirmação	Build, execução do Web e teste manual pelo navegador
2	Monitor COBOL	Prompt de Comando fixo mostrando as operações executadas pelo COBOL	Execução da API e teste de consulta/cadastro/atualização
3	Scripts PowerShell	Scripts para compilar, testar, ativar DB2 e validar fluxo completo	Execução do script testarfluxocompleto.ps1
4	Testes automatizados	Testes de Helper, formato fixo, API/Web e fluxo COBOL	dotnet test Cooperativa.slnx

Registro 1 - Criação da tela Razor Pages

Prompt utilizado:

Crie uma tela Razor Pages para o projeto da Cooperativa Financeira Alfa. A tela deve permitir consultar cliente, cadastrar cliente e atualizar contato. Ela deve chamar a API .NET, exibir o resultado da operação e, antes de cadastrar um cliente, abrir um popup confirmando nome, email e telefone.

Resposta obtida: A IA sugeriu estruturar a camada Web com Razor Pages, criando uma página de clientes com três formulários principais: consulta por código, cadastro de novo cliente e atualização de contato. Também sugeriu criar um serviço C# usando HttpClient para isolar a comunicação com a API. Para o cadastro, a IA sugeriu um botão de revisão que abre um

modal/popup com os dados preenchidos antes do envio definitivo.

Como foi usado no projeto: A sugestão foi usada para transformar o projeto em uma aplicação visual. Antes disso, o fluxo estava mais concentrado em API, COBOL, Helper e DB2. Com a tela Razor Pages, o sistema passou a ter uma interface acessível pelo navegador, permitindo demonstrar as operações sem depender apenas de terminal ou chamadas manuais de API.

Como isso ajudou a evoluir o projeto: Essa etapa evoluiu o projeto porque aproximou a solução de uma aplicação real. A tela facilitou a apresentação, tornou o fluxo mais compreensível para usuários não técnicos e mostrou que um sistema legado em COBOL pode ser acessado por uma interface moderna.

Como foi validado: A validação foi feita rodando a API, rodando o projeto Web e testando manualmente a consulta, o cadastro com popup de confirmação e a atualização de contato.

Registro 2 - Criação do Monitor COBOL

Prompt utilizado:

Quero que o processo do COBOL não apareça dentro da tela do site, mas sim em um Prompt de Comando separado. Esse prompt deve ficar aberto enquanto a aplicação estiver rodando e mostrar cada consulta, cadastro e atualização passando pelo COBOL.

Resposta obtida: A IA sugeriu criar um serviço hospedado na API para abrir uma janela de Prompt de Comando ao iniciar a aplicação. Essa janela ficaria acompanhando um arquivo de log em tempo real. A cada operação executada, o serviço responsável pelo fluxo COBOL escreveria no log os principais passos: entrada enviada para o COBOL, execução do programa, arquivo enviado ao Helper, retorno do Helper/DB2 e saída final para a API.

Como foi usado no projeto: Essa sugestão foi usada para criar um monitor de apresentação. O monitor fica aberto enquanto a API está rodando e registra as operações executadas pelo sistema. Ele ajuda a demonstrar que o COBOL está realmente sendo chamado dentro do fluxo e que o processamento passa pelo Helper e pelo DB2.

Como isso ajudou a evoluir o projeto: O monitor melhorou a apresentação do projeto. Em vez de apenas explicar verbalmente que o COBOL estava sendo executado, passou a ser possível mostrar visualmente cada operação acontecendo. Isso deixou o fluxo legado mais transparente e facilitou defender a arquitetura escolhida.

Como foi validado: A validação foi feita abrindo a API, usando a tela Web e observando o monitor registrar operações como CONSULTAR, CADASTRAR e ATUALIZAR.

Registro 3 - Scripts para rodar e testar a aplicação

Prompt utilizado:

Crie scripts PowerShell para facilitar a execução do projeto. Preciso automatizar build, testes, compilação do COBOL, configuração das variáveis do DB2 e execução do fluxo COBOL -> Helper -> DB2.

Resposta obtida: A IA sugeriu organizar scripts PowerShell para reduzir comandos manuais. Foram sugeridos scripts para testar a conexão com DB2, rodar o fluxo COBOL com Helper, limpar variáveis de ambiente e executar um teste completo, incluindo build da solution, testes automatizados, compilação COBOL e validação do fluxo com DB2.

Como foi usado no projeto: A sugestão foi usada para criar scripts de apoio ao desenvolvimento e à apresentação. O script de teste completo ajudou a validar várias partes do projeto em sequência, reduzindo o risco de esquecer alguma etapa manual.

Como isso ajudou a evoluir o projeto: Os scripts evoluíram o projeto porque trouxeram padronização para a execução. Em vez de rodar vários comandos soltos, o projeto passou a ter uma forma mais organizada de compilar, testar e verificar o fluxo completo. Isso também facilitou a preparação para a apresentação.

Como foi validado: A validação foi feita executando o script testarfluxocompleto.ps1 com usuário e senha do DB2, verificando se o build passava, se os testes rodavam, se o COBOL compilava e se a consulta retornava dados do banco.

Registro 4 - Apoio em testes automatizados

Prompt utilizado:

Crie testes automatizados para validar o projeto da Cooperativa Alfa. Quero testar o Helper em memória, o formato fixo de saída para o COBOL, a interpretação da saída final do COBOL, os endpoints da API e o serviço Web que chama a API.

Resposta obtida: A IA sugeriu testes com xUnit separados por responsabilidade. Foram sugeridos testes para o Helper, verificando consulta, cadastro, email duplicado e atualização; testes para o ResultadoHelper, validando o tamanho e formato fixo; testes para o FluxoCobolServico, validando a montagem da entrada e a interpretação da saída; testes para o controller da API; e testes para o serviço Web usando um HttpResponseMessageHandler fake.

Como foi usado no projeto: Parte dos testes automatizados foi criada com apoio da IA. As sugestões foram aproveitadas como base, mas precisaram de ajustes após a execução real, especialmente porque alguns testes inicialmente esperavam um formato diferente entre a saída do Helper e a saída final do COBOL.

Como isso ajudou a evoluir o projeto: Esse apoio ajudou a aumentar a confiabilidade do projeto. Os testes tornaram possível validar partes importantes sem depender apenas de testes manuais. Além disso, os erros encontrados nos testes ajudaram a ajustar melhor o formato de integração entre API, COBOL e Helper.

Como foi validado: A validação foi feita com dotnet test Cooperativa.slnx. Quando surgiram falhas, os testes e o código foram ajustados até que todos os testes passassem.

Cuidados tomados

As respostas da IA foram usadas apenas como apoio e não como entrega final pronta. Todas as sugestões aproveitadas foram testadas na prática, revisadas e ajustadas conforme o funcionamento real do projeto. Em vários pontos, foi necessário modificar o que a IA sugeriu para adaptar ao ambiente, à arquitetura escolhida e aos requisitos da entrega.

Conclusão

A Inteligência Artificial ajudou principalmente na organização, aceleração e refinamento do projeto. As sugestões geradas serviram como apoio durante o desenvolvimento, mas foram testadas, corrigidas e adaptadas ao ambiente real utilizado. Assim, a IA contribuiu como ferramenta auxiliar, enquanto a validação final foi feita com base na execução prática da aplicação, nos testes realizados e na integração real entre .NET, COBOL e DB2.